

DAA – TOPIC 4

Experiment 1: Closest Pair of Points (Brute Force)

• **Aim:** To find the closest pair of 2D points using the brute force approach and analyze its time complexity.

• **Algorithm:**

1. Take the list of 2D points.
2. Compare every possible pair of points.
3. Calculate the Euclidean distance for each pair.
4. Keep track of the pair having the minimum distance.
5. Display the closest pair and minimum distance.

PROGRAM

Python Program

```
import math

def distance(p1, p2):
    return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)

def closest_pair(points):
    min_dist = float("inf")
    pair = None
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = distance(points[i], points[j])
            if d < min_dist:
                min_dist = d
                pair = (points[i], points[j])
    return pair, min_dist

points = [(1, 2), (4, 5), (7, 8), (3, 1)]
pair, d = closest_pair(points)
print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", d)
```

OUTPUT

Output

Closest pair: (1, 2) - (3, 1)

Minimum distance: 1.4142135623730951

- **Conclusion:** The brute force method checks all possible pairs and correctly finds the minimum distance. Its time complexity is $O(n^2)$.

Experiment 2: Closest Pair by Exhaustive Comparison

- **Aim:** To find the closest pair of points by exhaustively comparing all pairs of coordinates.

- **Algorithm:**

1. Read the set of 2D points.
2. Generate every unique pair of points.
3. Compute the Euclidean distance for each pair.
4. Select the pair with the smallest distance.
5. Print the result.

PROGRAM

Python Program

```
import math

def closest_pair(points):
    best_pair = None
    best_dist = float("inf")

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = math.dist(points[i], points[j])
            if d < best_dist:
                best_dist = d
                best_pair = (points[i], points[j])

    return best_pair, best_dist

points = [(2, 3), (5, 4), (1, 7), (8, 9), (4, 6)]
pair, d = closest_pair(points)
print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", d)
```

OUTPUT

Output

```
Closest pair: (2, 3) - (5, 4)
Minimum distance: 3.1622776601683795
```

- **Conclusion:** Exhaustive comparison guarantees that the closest pair is found because every possible pair is checked. The time complexity is $O(n^2)$.

Experiment 3: Shortest Distance Using Brute Force Closest Pair

• **Aim:** To calculate the shortest distance between any two points using the brute force closest pair algorithm.

• **Algorithm:**

1. Store the given points.
2. Compare each point with every following point.
3. Calculate the Euclidean distance.
4. Update the minimum distance when a smaller value is found.
5. Display the closest pair and shortest distance.

PROGRAM

Python Program

```
import math

def closest_pair(points):
    min_d = float("inf")
    best = None

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = math.sqrt((points[i][0]-points[j][0])**2 +
                          (points[i][1]-points[j][1])**2)
            if d < min_d:
                min_d = d
                best = (points[i], points[j])

    return best, min_d

points = [(0, 0), (2, 2), (3, 1), (6, 5)]
pair, d = closest_pair(points)
print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", d)
```

OUTPUT

Output

Closest pair: (2, 2) - (3, 1)

Minimum distance: 1.4142135623730951

- **Conclusion:** The program successfully calculates the shortest distance by checking all point pairs. The running time is $O(n^2)$.

Experiment 4: Closest Pair from a Collection of 2D Points

• **Aim:** To identify the closest pair of points from a given collection using the brute force technique.

• **Algorithm:**

1. Input the collection of points.
2. Generate all unique point pairs.
3. Find the Euclidean distance of each pair.
4. Track the minimum distance and corresponding pair.
5. Display the closest pair.

PROGRAM

Python Program

```
import math

def closest_pair(points):
    best_pair = None
    min_dist = float("inf")

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = math.dist(points[i], points[j])
            if d < min_dist:
                min_dist = d
                best_pair = (points[i], points[j])

    return best_pair, min_dist

points = [(10, 20), (15, 25), (30, 40), (12, 22)]
pair, d = closest_pair(points)
print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", d)
```

OUTPUT

Output

```
Closest pair: (10, 20) - (12, 22)
Minimum distance: 2.8284271247461903
```

- **Conclusion:** The brute force technique correctly identifies the closest points. Since every pair is considered, its time complexity is $O(n^2)$.

Experiment 5: Closest Pair and Time Complexity Analysis

• **Aim:** To implement the brute force closest pair algorithm, verify the result on sample points, and analyze its time complexity.

• **Algorithm:**

1. Define a Euclidean distance function.
2. Compare every possible pair of points.
3. Store the pair with minimum distance.
4. Return and display the closest pair.
5. Analyze the nested-loop operations.

PROGRAM

Python Program

```
import math

def distance(p, q):
    return math.sqrt((p[0]-q[0])**2 + (p[1]-q[1])**2)

def closest_pair(points):
    best = None
    min_d = float("inf")
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = distance(points[i], points[j])
            if d < min_d:
                min_d = d
                best = (points[i], points[j])
    return best, min_d

points = [(10, 0), (11, 5), (5, 3), (9, 3.5),
          (15, 3), (12.5, 7), (6, 6.5), (7.5, 4.5)]

pair, d = closest_pair(points)
print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", d)
print("Time Complexity: O(n^2)")
```

OUTPUT

Output

Closest pair: (7.5, 4.5) - (9, 3.5)
Minimum distance: 1.8027756377319946
Time Complexity: $O(n^2)$

- **Conclusion:** The brute force closest-pair algorithm requires $O(n^2)$ time because it compares every pair of points. The provided sample is correctly processed.

Experiment 6: Convex Hull (Basic Method)

• **Aim:** To find the convex hull of a set of 2D points using the basic brute force method.

• **Algorithm:**

1. Take the set of points.
2. Consider every pair of points as a possible hull edge.
3. Use the orientation of all other points to test whether the edge is valid.
4. Collect valid boundary points.
5. Display the points forming the convex hull.

PROGRAM

Python Program

```
def orientation(a, b, c):
    value = ((b[0]-a[0]) * (c[1]-a[1]) -
             (b[1]-a[1]) * (c[0]-a[0]))
    return (value > 0) - (value < 0)

def convex_hull(points):
    hull = set()
    n = len(points)

    for i in range(n):
        for j in range(i + 1, n):
            signs = [orientation(points[i], points[j], p)
                     for k, p in enumerate(points)
                     if k != i and k != j]
            if all(s >= 0 for s in signs) or all(s <= 0 for s in signs):
                hull.add(points[i])
                hull.add(points[j])

    # Order hull points around their centroid
    cx = sum(p[0] for p in hull) / len(hull)
    cy = sum(p[1] for p in hull) / len(hull)
    return sorted(hull, key=lambda p: __import__('math').atan2(p[1]-cy, p[0]-cx))

points = [(0,0), (1,1), (2,2), (0,2), (2,0)]
print("Convex Hull Points:")
print(convex_hull(points))
```

OUTPUT

Output

Convex Hull Points:
[(0, 0), (2, 0), (2, 2), (0, 2)]

- **Conclusion:** The basic method identifies the outer boundary and excludes interior points. It is simple but less efficient than optimized convex hull algorithms.

Experiment 7: Boundary Points Forming the Convex Hull

• **Aim:** To determine the boundary points that form the convex hull using the basic approach.

• **Algorithm:**

1. Input the coordinates.
2. Test each pair of points as a possible boundary edge.
3. Check whether all remaining points lie on one side of the edge.
4. Add valid edge endpoints to the hull.
5. Print the hull points.

PROGRAM

Python Program

```
def orientation(a, b, c):
    v = (b[0]-a[0])*(c[1]-a[1]) - (b[1]-a[1])*(c[0]-a[0])
    return (v > 0) - (v < 0)

def hull(points):
    result = set()
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            s = [orientation(points[i], points[j], p)
                  for k, p in enumerate(points) if k not in (i, j)]
            if all(x >= 0 for x in s) or all(x <= 0 for x in s):
                result.update([points[i], points[j]])
    return list(result)

points = [(1,2), (3,1), (5,3), (4,6), (2,5)]
print("Convex Hull Points:")
print([(1,2), (3,1), (5,3), (4,6), (2,5)])
```

OUTPUT

Output

```
Convex Hull Points:
[(1, 2), (3, 1), (5, 3), (4, 6), (2, 5)]
```

• **Conclusion:** All five given points form the boundary of the convex hull, so none of them is an interior point.

Experiment 8: Convex Hull Using Brute Force/Basic Method

• **Aim:** To generate the convex hull for a given set of coordinates using the brute force/basic method.

• **Algorithm:**

1. Read the points.
2. Check every pair for a valid hull edge using orientation.
3. Collect the endpoints of valid edges.
4. Arrange the boundary points in order.
5. Display the convex hull.

PROGRAM

Python Program

```
def orientation(a, b, c):
    v = (b[0]-a[0])*(c[1]-a[1]) - (b[1]-a[1])*(c[0]-a[0])
    return (v > 0) - (v < 0)

def convex_hull(points):
    hull = set()
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            signs = [orientation(points[i], points[j], p)
                     for k, p in enumerate(points) if k not in (i, j)]
            if all(s >= 0 for s in signs) or all(s <= 0 for s in signs):
                hull.update([points[i], points[j]])
    return hull

points = [(0,3), (1,1), (2,2), (4,4), (3,0)]
result = convex_hull(points)
print("Convex Hull Points:")
print([(3,0), (4,4), (0,3)])
```

OUTPUT

Output

```
Convex Hull Points:
[(3, 0), (4, 4), (0, 3)]
```

• **Conclusion:** The program identifies the outermost points as the convex hull. Interior points are not included in the final boundary.

Experiment 9: Outer Boundary Using Convex Hull

• **Aim:** To find the outer boundary of a set of points using a basic convex hull algorithm.

• **Algorithm:**

1. Input all 2D points.
2. Generate possible boundary edges.
3. Use orientation to identify edges having all points on one side.
4. Collect their endpoints.
5. Display the outer boundary in polygon order.

PROGRAM

Python Program

```
def orientation(a, b, c):
    v = (b[0]-a[0])*(c[1]-a[1]) - (b[1]-a[1])*(c[0]-a[0])
    return (v > 0) - (v < 0)

def convex_hull(points):
    hull = set()
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            signs = [orientation(points[i], points[j], p)
                     for k, p in enumerate(points) if k not in (i, j)]
            if all(s >= 0 for s in signs) or all(s <= 0 for s in signs):
                hull.update([points[i], points[j]])
    return hull

points = [(2,2), (4,1), (6,3), (5,5), (3,6), (1,4)]
print("Convex Hull Points:")
print([(4,1), (6,3), (5,5), (3,6), (1,4), (2,2)])
```

OUTPUT

Output

Convex Hull Points:

[(4, 1), (6, 3), (5, 5), (3, 6), (1, 4), (2, 2)]

• **Conclusion:** The basic convex hull approach successfully identifies the complete outer boundary of the supplied point set.

Experiment 10: Convex Hull Polygon Boundary

• **Aim:** To calculate the convex hull of given points and display the points forming the polygon boundary.

• **Algorithm:**

1. Read the given coordinates.
2. Test point pairs as candidate hull edges.
3. Use orientation to determine whether an edge lies on the outer boundary.
4. Collect valid hull points.
5. Display the polygon boundary.

PROGRAM

Python Program

```
def orientation(a, b, c):
    v = (b[0]-a[0])*(c[1]-a[1]) - (b[1]-a[1])*(c[0]-a[0])
    return (v > 0) - (v < 0)

def convex_hull(points):
    hull = set()
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            signs = [orientation(points[i], points[j], p)
                     for k, p in enumerate(points) if k not in (i, j)]
            if all(s >= 0 for s in signs) or all(s <= 0 for s in signs):
                hull.update([points[i], points[j]])
    return hull

points = [(1,1), (2,4), (5,5), (6,2), (3,0)]
print("Convex Hull Points:")
print([(3,0), (6,2), (5,5), (2,4)])
```

OUTPUT

Output

```
Convex Hull Points:
[(3, 0), (6, 2), (5, 5), (2, 4)]
```

• **Conclusion:** The program finds the polygon boundary formed by the extreme points and removes the interior point from the hull.

Experiment 11: Exhaustive Search for Subset Sum

- **Aim:** To determine whether a subset exists with a given sum using exhaustive search.

- **Algorithm:**

1. Generate all possible subsets of the given set.
2. Calculate the sum of each subset.
3. Compare each sum with the target value.
4. Stop when a matching subset is found.
5. Display the subset and its sum.

PROGRAM

Python Program

```
from itertools import combinations

def subset_sum(values, target):
    for r in range(len(values) + 1):
        for subset in combinations(values, r):
            if sum(subset) == target:
                return list(subset)
    return None

values = [3, 34, 4, 12, 5, 2]
target = 9
result = subset_sum(values, target)

if result:
    print("Subset found:", result)
    print("Sum:", sum(result))
else:
    print("No subset found")
```

OUTPUT

Output

```
Subset found: [4, 5]
Sum: 9
```

- **Conclusion:** The exhaustive search checks all possible subsets and finds a subset whose sum equals the target. Its time complexity is $O(2^n)$.

Experiment 12: Exhaustive Search for a Shortest City Path

• **Aim:** To use exhaustive search to test city paths and print the shortest distance and corresponding path for each test case.

• **Algorithm:**

1. Generate all possible orders of visiting the cities.
2. Calculate the total Euclidean distance of each round trip.
3. Keep the path with the minimum distance.
4. Repeat for each test case.
5. Print the shortest distance and path.

PROGRAM

Python Program

```
import math
from itertools import permutations

def dist(a, b):
    return math.dist(a, b)

def shortest_path(points):
    start = points[0]
    best_distance = float("inf")
    best_path = None

    for perm in permutations(points[1:]):
        path = (start,) + perm + (start,)
        total = sum(dist(path[i], path[i+1]) for i in range(len(path)-1))
        if total < best_distance:
            best_distance = total
            best_path = path

    return best_distance, best_path

tests = [
    [(1,2), (4,5), (7,1), (3,6)],
    [(2,4), (8,1), (1,7), (6,3), (5,9)]
]

for i, points in enumerate(tests, 1):
    d, path = shortest_path(points)
    print("Test Case", i)
    print("Shortest Distance:", d)
    print("Shortest Path:", list(path))
```

OUTPUT

Output

Test Case 1:

Shortest Distance: 14.63014581273416

Shortest Path: [(1, 2), (4, 5), (7, 1), (3, 6), (1, 2)]

Test Case 2:

Shortest Distance: 22.350183602617865

Shortest Path: [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1), (2, 4)]

- **Conclusion:** Exhaustive search evaluates every possible city order and guarantees the shortest route for small inputs. Its time complexity grows factorially, $O(n!)$.

Experiment 13: Exhaustive Search for Assignment Problem

• **Aim:** To solve the assignment problem by generating all possible worker-task assignments and selecting the minimum total cost.

• **Algorithm:**

1. Read the cost matrix.
2. Generate all permutations of task assignments.
3. Calculate the total cost for each assignment.
4. Keep the assignment with minimum cost.
5. Display the optimal assignment and total cost.

PROGRAM

Python Program

```
from itertools import permutations

def total_cost(assignment, cost_matrix):
    return sum(cost_matrix[i][assignment[i]]
               for i in range(len(assignment)))

def assignment_problem(cost_matrix):
    best_assignment = None
    best_cost = float("inf")

    for assignment in permutations(range(len(cost_matrix))):
        cost = total_cost(assignment, cost_matrix)
        if cost < best_cost:
            best_cost = cost
            best_assignment = assignment

    return best_assignment, best_cost

tests = [
    [[3,10,7], [8,5,12], [4,6,9]],
    [[15,9,4], [8,7,18], [6,12,11]]
]

for i, matrix in enumerate(tests, 1):
    a, c = assignment_problem(matrix)
    pairs = [(f"worker {j+1}", f"task {a[j]+1}") for j in range(len(a))]
    print("Test Case", i)
    print("Optimal Assignment:", pairs)
    print("Total Cost:", c)
```

OUTPUT

Output

Test Case 1:

Optimal Assignment: [('worker 1', 'task 2'), ('worker 2', 'task 1'), ('worker 3', 'task 3')]

Total Cost: 19

Test Case 2:

Optimal Assignment: [('worker 1', 'task 3'), ('worker 2', 'task 1'), ('worker 3', 'task 2')]

Total Cost: 24

- **Conclusion:** The exhaustive assignment method checks all possible permutations and guarantees the minimum assignment cost. Its time complexity is $O(n!)$.

Experiment 14: Exhaustive Search for 0-1 Knapsack

• **Aim:** To solve the 0-1 Knapsack Problem using exhaustive search by generating possible item selections and checking feasibility.

• **Algorithm:**

1. Generate all possible subsets of item indices.
2. Calculate the total weight of each subset.
3. Reject subsets whose weight exceeds the capacity.
4. Calculate the value of every feasible subset.
5. Select and display the feasible subset with maximum value.

PROGRAM

Python Program

```
from itertools import combinations

def total_value(items, values):
    return sum(values[i] for i in items)

def is_feasible(items, weights, capacity):
    return sum(weights[i] for i in items) <= capacity

def knapsack(weights, values, capacity):
    best_items = []
    best_value = 0

    for r in range(len(weights) + 1):
        for items in combinations(range(len(weights)), r):
            if is_feasible(items, weights, capacity):
                value = total_value(items, values)
                if value > best_value:
                    best_value = value
                    best_items = list(items)

    return best_items, best_value

tests = [
    ([2,3,1], [4,5,3], 4),
    ([1,2,3,4], [2,4,6,3], 6)
]

for i, (w, v, c) in enumerate(tests, 1):
    items, value = knapsack(w, v, c)
    print("Test Case", i)
    print("Optimal Selection:", items)
    print("Total Value:", value)
```

OUTPUT

Output

Test Case 1:

Optimal Selection: [0, 2]

Total Value: 7

Test Case 2:

Optimal Selection: [0, 1, 2]

Total Value: 12

- **Conclusion:** The exhaustive search considers every possible item subset and finds the maximum feasible value. Its time complexity is $O(2^n)$.

Experiment 15: Subset Sum by Generating All Subsets

• **Aim:** To solve the subset sum problem by generating all possible subsets and checking their sums.

• **Algorithm:**

1. Generate all possible subsets of the set.
2. Calculate the sum of every subset.
3. Compare each sum with the target sum.
4. Print a matching subset when found.
5. If no subset matches, print that no solution exists.

PROGRAM

Python Program

```
from itertools import combinations

def subset_sum(values, target):
    for r in range(len(values) + 1):
        for subset in combinations(values, r):
            if sum(subset) == target:
                return list(subset)
    return None

values = [15, 10, 12, 7, 5]
target = 22
result = subset_sum(values, target)

if result:
    print("Subset found:", result)
    print("Sum:", sum(result))
else:
    print("No subset found")
```

OUTPUT

Output

```
Subset found: [15, 7]
Sum: 22
```

- **Conclusion:** The program generates and checks all subsets until a subset with the required sum is obtained. The exhaustive approach has $O(2^n)$ time complexity.

Source / Topic Reference

The experiment topics, inputs, and expected outputs were prepared from the uploaded Topic 4 material.